SlopBench: Informal Specs as In-Context Oracles for Secure Program Synthesis[1]

Jason Tang
Independent Researcher

With
Apart Research

Abstract

In secure program synthesis, specification elicitation has superseded code generation as the primary bottleneck [3, 1]. While executable oracles successfully restrict a model's "degrees of freedom" [4], formalizing subjective architectural quality to prevent unmaintainable "vibecoding slop" remains impractically expensive. We investigate whether informal engineering principles (e.g., the Zen of Python [10]) can function as zero-cost "in-context oracles" to enforce structural discipline. We introduce SlopBench, a 50-task benchmark curated to induce architectural degradation, and evaluate generated implementations using Claude Sonnet 4.6 [8]. Our empirical results demonstrate that informal oracles significantly collapse structural degrees of freedom: on highly complex tasks, conditioning reduced mean cyclomatic complexity by 40% (8.101 to 4.887, $p = 0.004$) and minimized nesting depth. Crucially, an adversarial control task confirms that oracles respond to specification semantics rather than generic brevity pressure: conditioning on a specification requiring explicit labeled branch variables increased cyclomatic complexity by +18.0 points. While a full 50-task scale evaluation reveals boundary conditions where aggressive conditioning can over-abstract simpler logic, soft oracles proved highly effective on complex structural challenges. To assess whether this structural rigor improves downstream agentic maintainability, we introduced a long-horizon maintenance stress test. Surprisingly, context-ablation revealed a null result: the frontier maintenance agent achieved near-perfect feature-implementation success regardless of whether it operated on oracle-constrained code, vibecoded slop, or an

[1]Research conducted prior to the SPS Fellowship, May 2026. Code and artifacts: github.com/davidkimai/specoracle.

empty context stub. This exposes a critical blind spot: because frontier models can currently brute-force localized architectural debt, standard pass/fail agentic evaluations actively mask structural slop. Ultimately, we demonstrate that informal specifications serve as highly effective, pragmatic soft oracles for enforcing code quality and auditability, bridging the gap between ambiguous engineering taste and secure synthesis at mere prompt cost.

1. Introduction

Current frontier language models excel at producing code that satisfies functional unit tests, effectively "solving" basic code generation benchmarks [6]. However, this functional success frequently masks a structural crisis: code that passes tests often manifests as deeply nested, monolithic, and unmaintainable "vibecoding slop." In the context of secure program synthesis (SPS), mathematical correctness guarantees and proof artifacts are insufficient if the resulting architecture is too structurally complex for a human to audit or safely maintain [1]. If a reviewer cannot quickly comprehend the control flow of a synthesized module, the system remains practically insecure.

Addressing this structural degradation is fundamentally a specification problem. As the SPS community increasingly recognizes, specification elicitation has superseded code generation as the primary bottleneck [2]. While it is theoretically possible to write strict, formal rules against architectural slop, complete formal specifications rarely exist in practice, and formalizing subjective architectural taste is impractically expensive [3]. We require a mechanism to constrain model outputs without the prohibitive overhead of formal verification.

Regehr [4] argues that executable oracles—such as fuzzers and property testers—are necessary to collapse an LLM's "degrees of freedom" and prevent the generation of poor work. While these "hard" oracles are highly effective for enforcing functional constraints, they are computationally heavy and difficult to apply to qualitative structural metrics. We propose investigating "soft" in-context oracles: Can informal, universally understood engineering principles (such as the Zen of Python [10]) collapse structural degrees of freedom merely through prompt-level conditioning?

To rigorously evaluate this, we must move beyond static generation tasks and measure how structural complexity impacts downstream modification. We introduce a methodology that measures both static architectural degradation and dynamic maintainability via long-horizon maintenance stress tests, recognizing that code structure ultimately only matters if it affects the ability of subsequent agents (or humans) to safely modify the software.

Our core contributions are as follows:

1. **SlopBench:** A 50-task pre-registered benchmark explicitly curated to induce architectural degradation, featuring long-horizon maintenance stressors, custom domain-specific specifications, and independently verifiable human references.

2. **SpecOracle:** An open-source evaluation pipeline integrating static analysis (Radon [11]), isolated Dockerized execution, and structured LLM-as-a-Judge scoring [7].

3. **SpecArena Context Ablation:** A rigorous downstream maintenance protocol designed to isolate the true impact of code structure on subsequent agentic modifications.

4. **Empirical Evidence:** Replicated results on Claude Sonnet 4.6 structured as a three-part experiment: a 20-task pilot demonstrating a 40% reduction in cyclomatic complexity ($p = 0.004$), an

adversarial control proving specification semantic specificity, and a full 50-task scale evaluation identifying structural boundary conditions.

2. Related Work

Secure Program Synthesis and the Specification Bottleneck. The transition from basic code generation to Secure Program Synthesis requires the simultaneous synthesis of software, specifications, and proofs [2]. However, as Dodds [3] identifies, specification elicitation has become the primary bottleneck; complete, coherent formal specifications are rarely available in real-world systems. Furthermore, attempting to formalize the highly subjective rules of architectural "taste" and maintainability is impractically expensive. SpecOracle is designed to operate within this pragmatic gap, abandoning the quest for formal architectural proofs in favor of utilizing informal engineering principles as zero-cost structural guardrails.

Executable Oracles and Degrees of Freedom. Regehr [4] posits that autonomous coding agents fail when granted the "degrees of freedom" to produce poor work, arguing that executable oracles (such as fuzzers and property testers) are required to pinch model outputs toward correctness. Our work directly tests the boundary conditions of this theory. While Regehr focuses on "hard" executable oracles, SpecOracle investigates the efficacy of "soft" in-context oracles. We aim to determine whether a non-executable, semantic constraint can still successfully collapse an LLM's structural degrees of freedom without the computational overhead of runtime validation machinery.

Spec-Conditioned Decoding vs. In-Context Conditioning. The SPS community has explored various methods to restrict model generation. For instance, Approximately Aligned Decoding [5] constrains LLM outputs directly at decode time, managing distribution distortion and ensuring constraint satisfaction. While powerful, this approach requires direct access to model weights or custom inference infrastructure. SpecOracle presents an ultra-accessible, application-layer alternative. By relying exclusively on in-context conditioning, we avoid manipulating logits and instead shape the context window, prioritizing qualitative structural metrics over hard constraint satisfaction.

Agentic Evaluation and Dynamic Maintainability. Our evaluation methodology builds upon the established code-generation tradition of HumanEval [6] and the qualitative LLM-as-a-Judge protocols inspired by MT-Bench [7]. However, we extend these static methodologies by introducing a dynamic, agentic evaluation component. Rather than assessing code structure merely as a static aesthetic via complexity analyzers like Radon [11], we empirically measure its friction against a downstream autonomous agent. Our downstream maintenance context-ablation test represents a novel approach to evaluating the true cost of architectural slop in agentic workflows.

3. Methods

Dataset: SlopBench. We introduce SlopBench, a novel 50-task public benchmark curated specifically to induce architectural degradation rather than merely testing functional logic. The benchmark spans a diverse 10-category taxonomy, encompassing advanced engineering patterns such as asynchronous generators, reliability-critical code paths, object lifecycle management, and complex data transformation pipelines. To rigorously evaluate the limits of soft oracles, we structure our evaluation into three parts: (1) a locked 20-task pilot split (`SlopBench-Min`) to measure efficacy on tasks known to induce high baseline complexity; (2) an adversarial control to isolate semantic adherence from code-shortening bias; and (3) a full 50-task scale evaluation to identify the boundary conditions and failure modes of soft oracles across diverse architectural patterns.

Generation Modes (Independent Variables). We frame our generation protocol as a rigorous control-treatment experiment. In the **Baseline (Control)** condition, the language model receives only the functional specification, representing standard unconstrained code generation ("vibecoding"). In the **Oracle (Treatment)** condition, the model receives the same prompt appended with a soft in-context constraint—either a universal engineering principle (the Zen of Python [10]) or a task-specific informal specification. To strictly verify that models respond to semantic structural constraints rather than merely defaulting to generating less code when a specification is present, the dataset includes an **Adversarial Control** task. This task utilizes a custom specification explicitly designed to *increase* cyclomatic complexity through mandated branch variables, effectively testing the bounds of in-context compliance.

Evaluation Pipeline (Dependent Variables). Generated modules are executed as untrusted code within a strictly isolated, network-disabled Docker sandbox. We separate our dependent variables into static structural metrics and dynamic maintainability.

- **Static Structural Metrics:** We utilize Radon [11] to deterministically measure Cyclomatic Complexity (CC), Nesting Depth, and the Maintainability Index of the generated Abstract Syntax Trees. A secondary LLM-as-a-Judge [7] scores qualitative adherence to the active informal specification.
- **Dynamic Maintainability (SpecArena):** To isolate the causal friction caused by architectural slop, we introduce a downstream context-ablation protocol. A maintenance agent is tasked with implementing a feature extension against three distinct context states: (A) the structurally degraded baseline code, (B) the oracle-conditioned clean code, and (C) an empty context stub. This allows us to empirically measure whether clean static architecture actually improves downstream agentic maintainability.

Reproducibility. The entire evaluation pipeline—from parallelized model generation to Dockerized static analysis and downstream context ablation—is fully automated via the open-source `specoracle` CLI tool. All evaluations enforce a temperature of 0.8 to ensure representative output distributions, and the codebase includes full runbooks to replicate the Claude Sonnet 4.6 evaluation locally.

4. Results

4.1 Act 1. SlopBench-Min Pilot: Strong Structural Effect (20 Tasks)

Our first experiment establishes that soft, in-context oracles are highly effective at enforcing architectural discipline on complex tasks. When conditioned on the Zen of Python or custom informal specifications over a 20-task pilot split, Claude Sonnet 4.6 demonstrated a significant 40% reduction in mean Cyclomatic Complexity (8.101 down to 4.887, paired Wilcoxon $p = 0.004$). Crucially, internal metrics reveal that the model did not simply "code golf" a denser monolithic script; nesting depth decreased (2.533 to 2.100) while function count increased (1.683 to 2.683). This indicates the oracle actively forced the model to modularize its logic, successfully collapsing the structural degrees of freedom.

This structural rigor was achieved with near-complete functional competence. The baseline achieved a 100.0% Pass@1 rate on the functional test suite, while the oracle-conditioned code achieved a 95.0% Pass@1 rate. The model remained capable of solving the logic; it simply wrote far more auditable code to do so.

4.2 Act 2. The Adversarial Control: Spec Semantic Specificity (Task 045)

The most frequent objection to soft structural oracles is that they merely act as a generic "write less code" pressure, rather than enforcing specific architectural intent. To definitively isolate semantic adherence,

Variant	Tasks	Samples	Pytest	Maint. P@1	Maint. P@3	Context P@1	CC Avg	Nesting	Fu
Vibecoded Baseline	20	3	100.0						
In-Context Oracle	20	3	95.0						
Human Reference	20	1	100.0						

Variant	CC Avg	Delta	Pytest
Vibecoded Baseline	1.000	-	100.0
In-Context Oracle	19.000	+18.000	100.0

we designed an adversarial control task (045_adversarial_spec). The specification explicitly required the use of explicitly labeled branch variables for every state transition, intentionally designing for a structurally verbose pattern.

As shown below, the model complied entirely. The oracle turned a simple dictionary lookup (Baseline CC = 1.0) into a massive cascade of 19 labeled branches (Oracle CC = 19.0), while maintaining a 100% functional pass rate. This definitively proves that the model is responding to the specific semantic content of the informal specification, rather than exhibiting a generic abbreviation bias.

4.3 Act 3. Full SlopBench: Scale and Boundary Conditions (50 Tasks)

When scaled to the full 50-task benchmark, the aggregate CC reduction weakened to a mean delta of -0.953 (Wilcoxon $p = 0.155$). Rather than a replication failure, this reveals a critical boundary condition. The 30 new tasks possessed much lower baseline complexity (mean CC $\sim$4.5). When code is already structurally simple, soft oracles face diminishing returns because there are fewer degrees of freedom to collapse. Furthermore, the task 045 adversarial control actively pulls the aggregate mean upwards by design.

4.4 Dynamic Maintainability: The Robustness of Frontier Models

While static metrics clearly demonstrate reduced slop on complex tasks, our SpecArena context-ablation test yielded a striking, scaled null result regarding dynamic maintainability. As shown in the Measurement Validity table across all 50 tasks, the maintenance agent successfully implemented the downstream feature extension 85.3% of the time when operating on structurally toxic baseline code. Remarkably, the agent achieved 86.0% success even when the context was completely ablated (an empty stub).

Ultimately, this null result provides a critical empirical insight for the SPS community: current frontier models exhibit high capability at zero-shot reasoning over small contexts, effectively brute-forcing their way through architectural slop. The true friction of technical debt, which critically impacts human reviewers, is currently masked in agentic evaluations.

Supplementary Metrics

5. Discussion and Limitations

Our empirical evaluation yields two critical insights for the Secure Program Synthesis community. First, we demonstrated that the specification bottleneck can be pragmatically mitigated using ”soft” in-context oracles. Informal engineering principles and domain-specific specifications successfully collapsed the structural degrees of freedom of a frontier language model, yielding a massive 40% reduction in cyclomatic complexity on complex tasks without the prohibitive cost of formal verification.

Variant	Tasks	Samples	Pytest	Maint. P@1	Maint. P@3	Context P@1	CC Avg	Nesting	Fu
Vibecoded Baseline	50	3	88.7						
In-Context Oracle	50	3	89.3						
Human Reference	50	1	100.0						

Variant	Real Context P@1	Stub Context P@1	Delta
Vibecoded Baseline	85.3		
In-Context Oracle	84.7		
Human Reference	88.0		

Second, our SpecArena context-ablation test revealed a notable characteristic of current frontier models: they are highly robust to architectural slop during localized, short-horizon maintenance tasks. Because the maintenance agent was able to perfectly patch the vibecoded slop, we cannot currently claim that clean architecture mathematically speeds up autonomous agents. However, this null result exposes a critical blind spot in how the industry evaluates code generation. If frontier models can mask the friction of technical debt during short-horizon agentic tasks, then "pass/fail" agentic evaluations are fundamentally insufficient for measuring secure architecture. If reviewers rely solely on whether an agent can pass the tests, architectural slop will silently accumulate until it exceeds the model's monolithic reasoning bounds, rendering the system practically insecure and entirely un-auditable by humans. Therefore, enforcing static structural discipline via soft oracles remains a mandatory defense-in-depth mechanism for long-term safety.

5.1 Limitations

We acknowledge several limitations in our current study. Most critically, our scale evaluation revealed an **Aggressive Conditioning Failure Mode**: on five of the advanced architectural tasks (e.g., circuit breakers, audit logs, event correlators), oracle conditioning caused a 100% failure rate on functional tests across all samples. The model adhered so strictly to structural decomposition that it over-abstracted and broke the underlying logic. This demonstrates the hard limits of soft oracles: overly aggressive structural conditioning can compromise functional correctness. Second, the evaluation is currently restricted to Python. Python's inherent emphasis on readability and the likely pre-training ubiquity of the "Zen of Python" might make in-context structural oracles artificially effective compared to their potential performance in denser, less semantically constrained languages like C++ or Rust. Finally, as noted above, our downstream context-ablation protocol yielded a null result, preventing us from making the strongest possible claim regarding the dynamic maintenance advantages of cleanly synthesized code.

5.2 Future Work

To find the exact threshold where an LLM's reasoning fails on toxic architecture, future iterations of SpecArena must move beyond localized short-term patching and introduce long-horizon tasks—complex, multi-file architectural refactoring requests where context limits strictly prevent brute-force reasoning. Additionally, executing the full 50-task SlopBench across a wider variety of newly released, frontier-class models will determine if "slop-robustness" is uniform across labs. Finally, future work should explore "Hybrid Oracles": combining the cheap, qualitative constraints of our in-context soft oracles with the rigorous, quantitative constraints of Regehr's executable hard oracles.

Variant	Tasks	Samples	MI	Judge
Vibecoded Baseline	20	3	72.370	7.800
In-Context Oracle	20	3	74.813	8.783
Human Reference	20	1	60.545	8.800

6. Conclusion

We investigated whether informal engineering principles could serve as zero-cost structural guardrails in the secure program synthesis pipeline. The results demonstrate that soft in-context oracles act as effective constraints, significantly reducing architectural slop and enforcing structural discipline without sacrificing functional correctness. While current frontier models are robust enough to brute-force short-term maintenance on degraded code—highlighting a dangerous blind spot in standard agentic evaluations—enforcing structural discipline at the prompt level remains a vital, zero-cost necessity for ensuring the long-term human auditability and security of synthesized software.

Code and Data

- **Code:** `github.com/davidkimai/specoracle`
- **Dataset:** `data/slopbench_min/` (20 tasks, YAML schema, pre-registered taxonomy in `data/DESIGN_NOTES`
- **Results:** `runs/claude_replicated/summary.csv` (140 rows, n=3, temperature=0.8)

Author Contributions

Jason Tang led project framing, benchmark design, implementation, evaluation, repository preparation, and paper scaffolding.

References

[1] **Dougherty, Q., and von Hippel, M.** 2026. *Lies, Damned Lies, and Proofs: Formal Methods are not Slopless*. LessWrong. Online.

[2] **von Hippel, M., Henniger, S., Dougherty, Q., and Miyazono, E.** 2026. *How to Solve Secure Program Synthesis*. LessWrong. Online.

[3] **Dodds, M.** 2025. *Specifications Don't Exist*. Galois Blog. Online.

[4] **Regehr, J.** 2026. *Zero-Degree-of-Freedom LLM Coding using Executable Oracles*. Online.

[5] **Melcer, D., et al.** 2024. *Approximately Aligned Decoding*. arXiv:2410.01103.

[6] **Chen, M., et al.** 2021. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374.

[7] **Zheng, L., et al.** 2023. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023. arXiv:2306.05685.

[8] **Anthropic.** 2026. *Claude Sonnet 4.6*. Online.

[9] **Wilcoxon, F.** 1945. *Individual comparisons by ranking methods*. Biometrics Bulletin.

[10] **Peters, T.** 2004. *The Zen of Python*. PEP 20. Online.

[11] **Radon.** 2026. *Python static analysis package*. Online.

LLM Usage Statement

LLM assistance was used for editing, organization, and repository packaging. Technical claims, commands, and metrics were checked against repository artifacts.